

P3. Reinforcement Learning

Andrés Río Nogués y Laia Beni

6 de diciembre, 2024



UNIVERSITAT^{DE}
BARCELONA

1 Introducción

El objetivo principal de esta práctica de IA es familiarizarnos con Q-learning, un algoritmo dentro del campo del aprendizaje por refuerzo (Reinforcement Learning o RL). Este se centra en el diseño y optimización de una política que permita a un agente ir desde un estado inicial hasta un estado terminal (ya sea conocido o no), maximizando la recompensa acumulada durante el proceso.

Para alcanzar este objetivo, emplearemos el enfoque del aprendizaje por refuerzo: un ciclo de prueba y error (trial-and-error), en el cual el agente ajusta sus estimaciones de recompensa en función de los resultados obtenidos. Este proceso iterativo continúa hasta que las recompensas aprendidas convergen hacia valores óptimos, indicando que el agente ha adquirido una política eficaz para resolver el problema planteado.

El algoritmo se basa en el uso de una tabla $Q(s,a)$ que registra los valores de recompensa esperados para cada acción a en un estado s . A través de interacciones con el entorno, el agente actualiza iterativamente estos valores usando la fórmula de aprendizaje Q .

2 Ejercicio 1 : Agente en un mundo pequeño

3				Goal
2				
1	Start			
	1	2	3	4

Figura 1: Nuestro "Mundo"

En este primer ejercicio implementaremos un escenario con un agente que tiene que alcanzar la casilla objetivo. Empezaremos formalizando el problema: Los estados de nuestro problema serán (formato (x,y)):

1. Primera columna: $(1,1)$, $(1,2)$, $(1,3)$
2. Segunda columna: $(2,1)$, $(2,3)$
3. Tercera columna: $(3,1)$, $(3,2)$, $(3,3)$
4. Cuarta columna: $(4,1)$, $(4,2)$, $(4,3)$

Acciones posibles de nuestro agente (el escenario visto en teoría no contempla ir hacia abajo):

1. Moverse Arriba $(x,y+1)$
2. Moverse Izquierda $(x-1,y)$
3. Moverse Derecha $(x+1,y)$

Empezando en el estado $(1,1)$ como estado inicial, el estado $(2,2)$ como un estado no accesible y el estado $(4,3)$ como nuestro estado objetivo. Para los estados imposibles, como la casilla $(2,2)$ o fuera del tablero, simplemente haremos que el agente nunca pueda acceder a ellos, es decir $x \subseteq [1-4]$, $y \subseteq [1-3]$ además de $(x,y) \neq (2,2)$

2.1 Primera prueba : Recompensas con valor -1

Definiremos nuestra matriz de recompensas con todas nuestras casillas con valor -1 y nuestro estado de recompensa con valor 100.

2.1.1 Imprime la primera, dos intermedias y la última Q-table. ¿Qué secuencia de acciones obtienes?

Q-table inicial :

	R	L	U
1,1	0.00	0.00	0.00
1,2	0.00	0.00	0.00
1,3	0.00	0.00	0.00
2,1	0.00	0.00	0.00
2,3	0.00	0.00	0.00
3,1	0.00	0.00	0.00
3,2	0.00	0.00	0.00
3,3	0.00	0.00	0.00
4,1	0.00	0.00	0.00
4,2	0.00	0.00	0.00
4,3	0.00	0.00	0.00

Tablero movimientos óptimos :

R	R	R	R
R		R	R
R	R	R	R

Figura 2: Tabla inicial

Q-table en el movimiento 70

	R	L	U
1,1	-0.94	0.00	-0.41
1,2	0.00	0.00	4.72
1,3	21.29	0.00	0.00
2,1	-0.91	-0.76	0.00
2,3	53.99	-0.66	0.00
3,1	1.29	-0.30	-0.30
3,2	-0.30	0.00	22.16
3,3	91.76	1.75	0.00
4,1	0.00	-0.30	7.59
4,2	0.00	-0.30	51.00
4,3	0.00	0.00	0.00

Tablero movimientos óptimos :

R	R	R	R
U		U	U
L	U	R	U

Figura 3: Tabla movimiento 70

Q-table en el movimiento 140

	R	L	U
1,1	-0.96	0.00	35.96
1,2	0.00	0.00	57.45
1,3	75.83	0.00	0.00
2,1	-0.59	-0.76	0.00
2,3	87.46	52.63	0.00
3,1	5.67	-0.30	-0.30
3,2	-0.30	0.00	22.16
3,3	99.77	22.65	0.00
4,1	0.00	-0.30	30.59
4,2	0.00	-0.30	75.99
4,3	0.00	0.00	0.00

Tablero movimientos óptimos :

R	R	R	R
U		U	U
U	U	R	U

Figura 4: Tabla movimiento 140

Q-table final

	R	L	U
1,1	60.66	0.00	62.17
1,2	0.00	0.00	70.19
1,3	79.10	0.00	0.00
2,1	69.55	35.84	0.00
2,3	89.00	70.19	0.00
3,1	78.91	36.07	43.14
3,2	-0.30	0.00	85.15
3,3	100.00	79.10	0.00
4,1	0.00	50.91	88.92
4,2	0.00	49.72	99.99
4,3	0.00	0.00	0.00

Tablero movimientos óptimos :

R	R	R	R
U		U	U
U	R	R	U

Numero iteraciones necesarias para aprender : 206

Figura 5: Tabla final

Con los valores pensados en el apartado siguiente 2.1.2 de $\alpha = 0.3$, $\epsilon = 0.4$ y $\gamma = 0.9$, podemos ver como en el movimiento número 70 de nuestro agente ya tiene una política que es igual a la óptima de inicio a fin, pero no es hasta el movimiento 206 que acaba de actualizarse los valores. La secuencia obtenida es de arriba, arriba, derecha, derecha y derecha, llegando al final con 5 movimientos.

2.1.2 Después de probar un poco, ¿cuáles son los valores que elegiste para los parámetros alpha, gamma y epsilon? ¿Por qué?

Para elegir los valores de estos 3 parámetros (que oscilan entre 0 y 1), primero necesitamos entender qué significan y cómo afectan a nuestro algoritmo. Vamos a explicarlo aquí en detalle para que en los siguientes ejercicios usemos un razonamiento similar.

Alpha (Tasa de aprendizaje):

Un valor de alpha más alto significa que damos más peso a la información más reciente, mientras que un alpha más bajo significa que la información pasada y presente tienen más importancia. Al decir esto, estamos indicando que establecer un alpha bajo es más "consistente", ya que consideramos toda la información acumulada del pasado. Por lo tanto, en la mayoría de los casos, al elegir un alpha bajo garantizamos un mejor rendimiento en la convergencia. No obstante, el problema es que cuanto más bajo lo establecemos, más tiempo se tarda en converger, ya que actualizamos la información más lentamente. Después de probar un poco, establecimos que un valor de 0.3 funciona bien para nuestro algoritmo.

Epsilon (Exploración-Explotación):

Epsilon controla el equilibrio entre la exploración y la explotación. Durante nuestro entrenamiento, podemos explorar nuevas acciones (con probabilidad epsilon) o explotar el conocimiento actual (con probabilidad de $1 - \epsilon$). En las clases teóricas, hemos visto que con suficiente exploración, nuestro algoritmo converge a la solución óptima. Esto se debe a que, en primer lugar, necesitamos explorar los estados para saber si son estados buenos o malos. Una vez que hemos explorado lo suficiente, deberíamos explotar lo que hemos aprendido y tratar de encontrar la solución óptima, por lo que será el momento de la explotación.

Por lo tanto, nuestra estrategia será comenzar con un epsilon alto para fomentar la exploración y reducirlo gradualmente con el tiempo a medida que el algoritmo aprende más sobre el entorno. Para simplificar nuestro trabajo, vamos a usar un epsilon estático de 0.4 para que el 60% de las veces aplique el conocimiento anterior. Al ser un espacio muy reducido con acciones muy limitadas, la probabilidad de que una acción aleatoria sea la óptima no es baja. En escenarios más complejos ir actualizando la epsilon a medida que se explore lo suficiente es mejor para no explorar tanto una vez tengamos conocimiento suficiente. Por eso preferimos simplificar nuestro caso con el estático.

Gamma (Factor de descuento):

Un gamma alto prioriza las recompensas a largo plazo, mientras que un gamma bajo prioriza las recompensas a corto plazo. Por lo tanto, si nuestro problema involucra una secuencia de acciones a largo plazo, usamos un gamma alto para dar prioridad a las recompensas futuras. Al elegir un gamma más alto, aseguramos tener en cuenta la recompensa del estado objetivo. En este caso, dado que todos los estados tienen la misma recompensa de -1, excepto el objetivo que tiene 100, debemos elegir un valor alto de gamma porque necesitamos llegar al estado objetivo para obtener la recompensa deseada (largo plazo). Un valor muy cercano a 1 hace que se centre solo en recompensas futuras y puede llegar a realentizar la convergencia. Por lo tanto, elegimos 0.9 para gamma.

Los valores anteriores se eligen para garantizar que el algoritmo converja a la solución óptima. No tienen porque converger a la solución óptima de la mejor manera.

2.1.3 ¿Cómo evaluas la convergencia del algoritmo? ¿Cuánto tiempo tarda en converger?

Diremos que nuestro algoritmo ha convergido en el momento que la q-tabla no cambia después de varios episodios. En nuestro código cada vez que llega al estado objetivo miramos el valor de la q-tabla y lo restamos con la última vez que llego al objetivo. Si su diferencia es menor a un valor establecido (en nuestro caso 10^{-12}) podemos asegurar que no ha aprendido nada en ese episodio y podemos detener la ejecución. Para estar más seguros de que no se haya detenido por alguna casualidad, contamos las veces seguidas que tiene que suceder esto y ya paramos. En nuestro caso, la diferencia de q-tables tiene que dar 10^{-12} durante 10 episodios consecutivos. Al tener una probabilidad de por medio, la convergencia depende de la ejecución. En nuestro caso del apartado 2.1.1 obtenemos 206 episodios, aunque este tiene una media aproximada de 400 (visto más adelante en el punto 2.2.2)

2.2 Segunda prueba : Recompensas con una función de distancia

Ahora en vez de utilizar una matriz de recompensas con todos los valores asignados a -1, utilizaremos una función que determina la distancia a la meta.

3	-3	-2	-1	100
2	-4		-2	-1
1	-5	-4	-3	-2
	1	2	3	4

Figura 6: Recompensas con una función de distancia

Utilizamos la distancia de manhattan para dar una recompensa diferente al agente. Esta le da una recompensa mejor al agente en función de como de cerca se encuentre al final. Si este se aleja será más castigado a si este se acerca.

2.2.1 Imprime la primera, dos intermedias y la última Q-table. ¿Qué secuencia de acciones obtienes?

Q-table inicial :

	R	L	U
1,1	0.00	0.00	0.00
1,2	0.00	0.00	0.00
1,3	0.00	0.00	0.00
2,1	0.00	0.00	0.00
2,3	0.00	0.00	0.00
3,1	0.00	0.00	0.00
3,2	0.00	0.00	0.00
3,3	0.00	0.00	0.00
4,1	0.00	0.00	0.00
4,2	0.00	0.00	0.00
4,3	0.00	0.00	0.00

Tablero movimientos óptimos :

R	R	R	R
R		R	R
R	R	R	R

Figura 7: Tabla inicial

Q-table en el movimiento 70

	R	L	U
1,1	-3.33	0.00	-2.15
1,2	0.00	0.00	10.16
1,3	36.28	0.00	0.00
2,1	-3.00	-1.50	0.00
2,3	69.39	0.00	0.00
3,1	-1.02	-2.04	1.09
3,2	22.90	0.00	0.00
3,3	94.24	12.82	0.00
4,1	0.00	0.00	13.26
4,2	0.00	0.00	75.99
4,3	0.00	0.00	0.00

Tablero movimientos óptimos :

R	R	R	R
U		R	U
L	U	U	U

Figura 8: Tabla movimiento 70

Q-table en el movimiento 140

	R	L	U
1,1	-3.84	0.00	38.80
1,2	0.00	0.00	58.91
1,3	75.02	0.00	0.00
2,1	-2.71	3.39	0.00
2,3	87.98	34.33	0.00
3,1	-1.02	-2.04	6.34
3,2	36.25	0.00	0.00
3,3	99.84	29.97	0.00
4,1	0.00	0.00	13.26
4,2	0.00	0.00	83.19
4,3	0.00	0.00	0.00

Tablero movimientos óptimos :

R	R	R	R
U		R	U
U	L	U	U

Figura 9: Tabla movimiento 140

Q-table final

	R	L	U
1,1	36.35	0.00	56.56
1,2	0.00	0.00	67.29
1,3	78.10	0.00	0.00
2,1	6.39	45.62	0.00
2,3	89.00	67.28	0.00
3,1	-1.02	-2.04	29.98
3,2	64.23	0.00	0.00
3,3	100.00	78.10	0.00
4,1	0.00	0.00	13.26
4,2	0.00	0.00	94.24
4,3	0.00	0.00	0.00

Tablero movimientos óptimos :

R	R	R	R
U		R	U
U	L	U	U

Numero iteraciones necesarias para aprender : 146

Figura 10: Tabla final

Usando los mismos parámetros de antes obtenemos una política óptima en el episodio 146, con secuencia de arriba , arriba , derecha, derecha y derecha.

2.2.2 Después de probar un poco, ¿cuáles son los valores que elegiste para los parámetros alpha, gamma y epsilon? ¿Por qué?

Ahora que nuestra función de recompensa es diferente y nos da información de como de cerca estamos de nuestro objetivo, podemos aprovechar esta información a nuestro favor. Recordemos que un gamma más alto se centra en las recompensas a largo plazo, y viceversa. En este caso, podemos dar más importancia a las recompensas a corto plazo, porque ya nos están proporcionando información crucial. De hecho, al estar en un cierto estado, ya podemos saber si los siguientes estados son buenos o malos. Eso significa que no necesitamos llegar al estado objetivo para determinar la efectividad de un estado dado. En consecuencia, podemos establecer nuestro gamma en un valor más bajo. Vamos a fijar gamma en 0.6 y ver los resultados. Probar con valores gamma tan tan bajos como 0.3 o 0.01 no mejorará el código porque solo tendrá en cuenta la recompensa inmediata, sin valorar el valor del estado futuro.

```
Media de episodios de usar recompensa 1 = 369.308
Media de episodios de usar recompensa heuristica = 366.658
Media de episodios de usar recompensa heuristica con menos gamma= 357.815
```

Figura 11: Media de 1000 iteraciones

Ejecutando 1000 veces los diferentes escenarios podemos ver como la media baja, viendo como una función de recompensa más sofisticada nos ayuda a reducir el número de episodios. Respecto a la alpha y el epsilon los podemos mantener debido a que no influye el tipo de recompensa usado.

2.2.3 ¿Cómo evalúas la convergencia del algoritmo? ¿Cuánto tiempo tarda en converger?

Utilizamos exactamente el mismo proceso a antes, explicado en el 2.1.3. Sigue con valores entre 300-400 episodios.

2.2.4 ¿Cuál es el efecto de la nueva función de recompensa?

Podemos ver que en el escenario anterior, todos los estados tenían la misma recompensa de -1, excepto el estado objetivo. La información que tenemos con este sistema de recompensas es que todos los estados con recompensas negativas son estados malos.

Sin embargo, en este segundo caso, tenemos un sistema de recompensas más sensible. Esta vez, podemos ver que los valores negativos no solo nos dicen que son estados malos, sino que también indican como de malos son. Cuanto menor sea la recompensa, peor será el estado. En consecuencia, dado que nuestros valores Q se actualizan a través de las recompensas, si las recompensas nos proporcionan más información, el algoritmo funcionará mejor. Como hemos visto ya en el apartado 2.2.2, el cambio de recompensa ya tiene una mejora asociada y si encima mejoramos nuestra gamma, mejoramos aun más respecto al escenario anterior.

2.2.5 ¿Cómo se relaciona esto con los algoritmos de búsqueda estudiados en P1? ¿Podrías aplicar alguno de ellos en este caso?

Podemos relacionarlo con el algoritmo A* y el uso de las heurísticas. Los sistemas de recompensa son comparables a las heurísticas ya que son indicadores de como de cerca estamos a un estado objetivo. El algoritmo A* se podría utilizar perfectamente para resolver este problema, ya que el objetivo es alcanzar el estado objetivo dado un estado inicial. Podemos considerarlo como un escenario determinista porque las acciones decididas por el agente se llevarán a cabo, no hay aleatoriedad que afecte las acciones. Con nuestro sistema de recompensa sería suficiente para que el algoritmo A* encuentre la solución óptima.

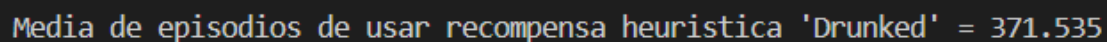
2.3 Tercera prueba : Drunken sailor

En esta última prueba introduciremos aleatoriedad para nuestro agente. Ahora al realizar un movimiento, tendrá un 99% de probabilidad de que realice ese movimiento. El otro 1% hará que el agente se mueva en otra dirección aleatoria. Para implementar esto lo que haremos será dejar que el agente elija su acción y luego comprobar si esta "borracho" o no. Si se cumple la probabilidad de "borracho" cambiaremos su acción a una aleatoria. Para la recompensa usaremos la heurística que se basa en la distancia.

2.3.1 ¿Cuál es tu elección de parámetros? ¿Por qué?

Utilizaremos los parámetros usados en el punto 2.2.2 al ser mejor que las otras probadas y así podemos comparar con más facilidad. Al ser un entorno muy pequeño y con solo 3 acciones posibles vamos a dejar los parámetros iguales para que la comparación sea más fácil. Realmente estaría bien ajustar estos parámetros en escenarios más grandes para ayudar a la convergencia. Más adelante en el escenario del ajedrez veremos que cambios hacer a cada parámetro para tolerar mejor la incertidumbre. Ahora mismo el efecto que esperamos es que el tiempo de convergencia sea algo mayor debido a la toma de decisiones con menos criterio.

2.3.2 Asumiendo que el marinero se encuentra en un estado que permite el aprendizaje: ¿cuántas noches de borrachera son necesarias para que domine el peligroso camino hacia la cama? Compáralo con el escenario anterior, determinista.



Media de episodios de usar recompensa heuristica 'Drunked' = 371.535

Figura 12: Media de 1000 iteraciones

Ejecutando el código "Drunked" 1000 veces, observamos que el marinero necesita una media de 371 episodios para aprender el camino óptimo, aproximadamente 15 episodios más que en el escenario determinista. Si consideramos que cada noche realiza un intento para llegar a su cama, esto significa que necesitaría 371 noches de aprendizaje en promedio.

La introducción de la aleatoriedad hace que el agente tarde más en converger, pero el aumento no es significativo debido a que el escenario es pequeño y tiene pocas acciones posibles. La aleatoriedad afecta los resultados al introducir la posibilidad de que el agente no ejecute la acción elegida, lo que dificulta la explotación de su conocimiento. Como resultado, el agente explora más, incluso cuando ya dispone de suficiente información para encontrar el camino óptimo.

En contraste, en el escenario determinista, el agente puede centrarse antes en explotar la política aprendida, lo que permite una convergencia más rápida y eficiente.

2.3.3 ¿Cuál es el camino óptimo encontrado? Si observamos al agente intentar seguirlo, ¿siempre tomaría el mismo camino?

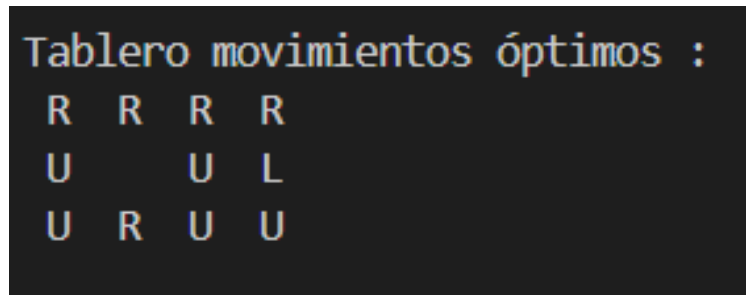


Figura 13: Movimientos óptimos encontrados

El óptimo encontrado es el mismo que en el ejercicio anterior. Sin embargo, si dejamos que el marinero intente seguirlo, es posible que no siga exactamente el mismo camino que se muestra en la imagen. Esto se debe al 99% de precisión para realizar el movimiento correcto. Por ejemplo, si comenzamos en la primera posición (1,1), según la política, debemos movernos hacia arriba como la mejor opción, pero el marinero podría moverse a la derecha.

2.3.4 ¿Podríamos aplicar alguno de los algoritmos de P1 aquí? ¿Por qué? Pista: Piensa en las nociones de determinismo vs aleatoriedad y de camino vs política

No podríamos usar los algoritmos de la práctica 1 debido que el algoritmo A* se utiliza para casos deterministas.

En este caso, el del marinero borracho, estamos involucrados con valores y factores aleatorios, lo que impide tener una resolución segura. Por lo tanto, los algoritmos del P1 no serían útiles ni obtendrían el resultado deseado para este caso.

Además, en los casos deterministas obtenemos un camino para alcanzar nuestro objetivo final, pero en casos aleatorios como este, queremos encontrar una política que nos indique el mejor movimiento a realizar en un estado determinado. Esto se debe a que la segunda situación no es determinista, por lo que no podemos asegurar cuál será nuestro próximo estado. Por lo tanto, no tiene sentido hablar de un camino fijo, ya que es posible que en cualquier momento nos salgamos de este camino y que tengamos que adaptar nuestra ruta.

3 Ejercicio 2 : RL aplicado al ajedrez

En este ejercicio miraremos de implementar el Q-learning en una partida de ajedrez. Usaremos una posición ya empezada, que tiene los 2 reyes y una torre blanca colocados de la siguiente manera:

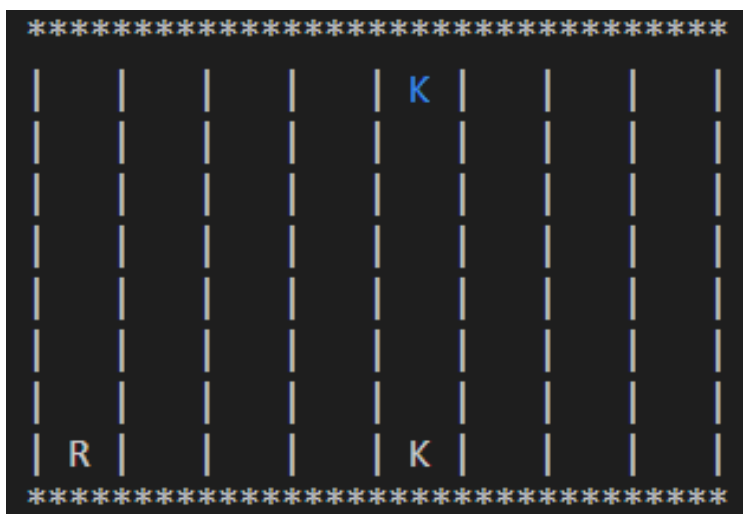


Figura 14: Nuestro tablero de ajedrez

Empiezan las blancas y buscaremos el camino óptimo para llegar al jaque mate teniendo en cuenta que el jugador negro no hará ningún movimiento. Consideramos una política como óptima si esta llega a cualquier estado de jaque mate en la menor cantidad de movimientos, o en este escenario concreto, una política que use solamente 6 movimientos (1 de la torre a la última fila y 5 del rey para colocarse enfrente del rey negro).

3.1 Adaptar el Q-learning con recompensa fija de -1

Ahora en vez de tener una qtable con un diccionario con keys del formato (estado, accion) = qvalue ahora tenemos un diccionario con diccionarios con el formato [estado][estadofuturo] = qvalue

3.1.1 ¿Qué secuencia de acciones obtienes?

```
Sequence of moves: [[[0, 4, 12], [2, 4, 6], [0, 0, 2]], [[6, 4, 6], [7, 0, 2]], [[5, 5, 6], [7, 0, 2]], [[4, 5, 6], [7, 0, 2]], [[3, 5, 6], [7, 0, 2]], [[2, 4, 6], [7, 0, 2]], [[2, 4, 6], [0, 0, 2]]]
```

Figura 15: Secuencia de movimientos

Podemos ver que el rey es la primera pieza en moverse, y no la torre. Esto se debe al sistema de recompensas que configuramos inicialmente. Dado que todos los estados tenían una recompensa de -1, nuestro algoritmo no puede distinguir la diferencia entre mover primero el rey o mover primero la torre.

3.1.2 Después de probar un poco, ¿cuáles son los valores que elegiste para los parámetros alpha, gamma y epsilon? ¿Por qué?

Siguiendo la explicación del apartado 2.1.2 usaremos unos parámetros similares. El único cambio será que ahora no aplicaremos un epsilon estático, sino uno dinámico. Así al principio aseguramos una exploración más amplia y una vez haya explorado lo suficiente, explotamos. Empezamos con un epsilon de 0.9 y lo iremos bajando hasta llegar a un epsilon de 0.1. Otra solución con un epsilon estático puede llegar en menos mates a una política. Si por ejemplo aplicamos directamente una epsilon de 0.1, al no tener nada de información al principio, la explotación se convierte en una exploración. Este método consigue converger con apenas

100-200 movimientos, el problema es que es menos estable y no suele llegar a la política óptima. Además de que suele tardar más tiempo en llegar al mate. Por esa razón seguiremos con la metodología tradicional de ir reduciendo el epsilon para el resto de pruebas (Asi conseguimos resultados más estables para las pruebas).

3.1.3 ¿Cómo evaluas la convergencia del algoritmo? ¿Cuánto tiempo tarda en converger?

Ahora para ver la convergencia vemos todas las diferencias de valores antiguos y nuevos en la qtable y cogemos la máxima. Si al acabar el episodio esta diferencia máxima supera un umbral 5 veces consecutivas implica que ya ha encontrado la política. El umbral escogido es de 0.1, lo que indica que si en todo el episodio el cambio más significativo de la tabla no es superior a 0.1 significa que ya ha encontrado una política. Para asegurarnos que no es un caso extremo miramos que no se cruce el umbral en 5 episodios consecutivos. Para ver la media de tiempo que tarda en converger ejecutaremos 5 veces nuestro código y haremos la media de episodios (o jaques mates) necesarios para encontrar la política. El resultado obtenido es :

```
stating AI chess...
Iteracion 0 Episodios = 826
stating AI chess...
Iteracion 1 Episodios = 872
stating AI chess...
Iteracion 2 Episodios = 874
stating AI chess...
Iteracion 3 Episodios = 869
stating AI chess...
Iteracion 4 Episodios = 780

Media de episodios en 5 iteraciones : 844.2
```

Figura 16: Media de episodios

Podemos ver como necesita unos 850 mates de media para llegar a la política.

3.2 Probando la heurística como recompensa

Para este caso hemos modificado la heurística de la P1. Hemos creado una muy básica que penaliza movimientos no deseados y los que si queremos los penalizamos muy poco. Primero hacemos que el mate devuelva 100, igual que antes. Si no es mate, vamos sumando valores negativos en función del proceso del blanco. Para la torre restamos mucho si no se mueve (-50), si va a la posición de jaque no restamos nada , si se mueve a la última fila restamos muy poco (-10) y si hace otro movimiento que no es ir a la última fila penalizamos bastante (-200). Para el rey penalizamos en función de la fila que este (más lejos más penalización) y en función de la columna (más lejos de la columna del rey rival más penalización).

3.2.1 ¿Qué secuencia de acciones obtienes?

```
Sequence of moves: [[[0, 4, 12], [2, 4, 6], [0, 0, 2]], [[6, 4, 6], [7, 0, 2]], [[5, 5, 6], [7, 0, 2]], [[4, 5, 6], [7, 0, 2]], [[3, 5, 6], [7, 0, 2]], [[2, 4, 6], [7, 0, 2]], [[2, 4, 6], [0, 0, 2]]]
```

Figura 17: Media de episodios

Obtenemos la misma secuencia de movimientos a la anterior.

3.2.2 Después de probar un poco, ¿cuáles son los valores que elegiste para los parámetros alpha, gamma y epsilon? ¿Por qué?

Como hemos dicho previamente en el apartado 2.2.2, al tener un sistema de recompensa que nos da información de como estamos explorando, bajar el gamma nos puede favorecer. Asi que para este escenario volvemos a aplicar la misma alpha y el mismo epsilon, pero bajamos la gamma a 0.6.

3.2.3 ¿Cómo evalúas la convergencia del algoritmo? ¿Cuánto tiempo tarda en converger?

Usamos la misma metodología del apartado anterior. Volvemos a ejecutar 5 veces para ver la media de episodios y además comparamos la diferencia de usar gammas diferentes.

```
Media de episodios en 5 iteraciones : 845.0

Media de episodios en 5 iteraciones (gamma mas bajo): 800.6
```

Figura 18: Media de episodios con gamma 0.9 y gamma 0.6

Se puede observar como el número de episodios se ha reducido solamente al aplicar una gamma más adecuada.

3.2.4 ¿Cuál es el efecto de esta nueva recompensa?

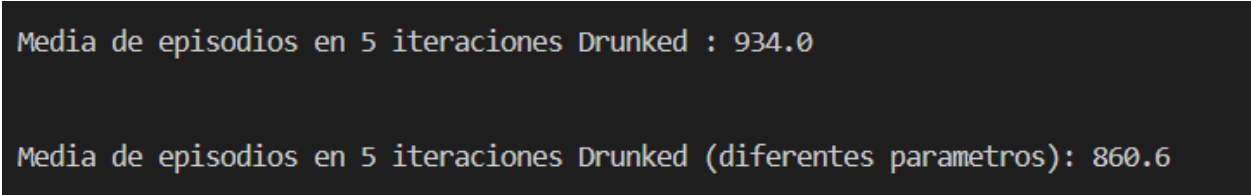
Aprovechando la nueva información que nos da la heurística podemos ver como el número de episodios necesarios para converger es menor, ayudando a nuestro agente. Además conseguimos que ahora la torre haga el primer movimiento, priorizando el jaque, antes que cualquier movimiento del rey.

3.3 Drunken Sailor jugando al ajedrez

Para añadir aleatoriedad al código lo que haremos es que después de tomar la acción, aplicaremos una probabilidad del 1%. Si sale este porcentaje, haremos que el agente cambie su acción a una aleatoria, independientemente de lo que haya elegido. Para este test elegiremos la recompensa de -1 y 100.

3.3.1 ¿Cuál es tu elección de parámetros? ¿Por qué?

Elegiremos dos sets. El primero serán los parámetros del apartado de la recompensa de -1 y 100 para poder compararlo con más facilidad. El segundo serán unos parámetros ajustados debidos a la aleatoriedad. En este segundo reduciremos el alpha de 0.3 a 0.15 para suavizar las actualizaciones y ser menos sensible a posibles ruidos y reduciremos el gamma de 0.9 a 0.7 para que se centre algo más en las recompensas inmediatas, sabiendo que tiene un futuro incierto e impredecible. La epsilon seguirá igual. Viendo los resultados obtenemos :



```
Media de episodios en 5 iteraciones Drunked : 934.0
```

```
Media de episodios en 5 iteraciones Drunked (diferentes parametros): 860.6
```

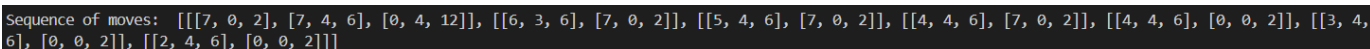
Figura 19: Media de episodios Drunked y con diferentes parámetros

Podemos ver como ajustar los parámetros si que supone una mejora clara a la hora de aprender. Al reducir el ruido y el ritmo de aprendizaje nuestro marinero borracho aprende de forma más rápida al no tener en cuenta tanto los posibles movimientos aleatorios que tenga.

3.3.2 Suponiendo que nuestro marinero obsesivo está en un estado que permite el aprendizaje: ¿cuántas partidas tiene que jugar antes de estar satisfecho de haber encontrado la mejor estrategia y poder irse a la cama? Compáralo con el escenario anterior, determinista.

Viendo ambos resultados, el determinista del punto 3.1.3 con media de 844 noches para aprender y el borracho del punto 3.3.1 que con los mismos parámetros tarda 934 noches, podemos ver como claramente el entorno determinista aprende mucho antes. Si se ajustan los parámetros podemos conseguir reducir esta diferencia a tan solo 860 noches con el marinero borracho, pero igualmente si el marinero tiene la capacidad completa de elegir sus decisiones aprenderá antes que si va borracho y no tiene control absoluto.

3.3.3 ¿Cuál es el camino óptimo encontrado? Si observáramos al marinero intentar seguirlo, ¿seguiría siempre el mismo camino?



```
Sequence of moves: [[[7, 0, 2], [7, 4, 6], [0, 4, 12]], [[6, 3, 6], [7, 0, 2]], [[5, 4, 6], [7, 0, 2]], [[4, 4, 6], [7, 0, 2]], [[4, 4, 6], [0, 0, 2]], [[3, 4, 6], [0, 0, 2]], [[2, 4, 6], [0, 0, 2]]]
```

Figura 20: Secuencia del marinero borracho con los parámetros iguales al determinista

Si el marinero intenta seguir este camino, al estar borracho no tiene porque seguirlo. Existe siempre ese 1% de probabilidad de que se desvie y tenga que tomar otro camino no óptimo.

3.4 Comparación entre los dos escenarios

3.4.1 Compara la aplicación de Q-learning en este escenario de ajedrez con la de la cuadrícula del ejercicio 1. ¿En qué se diferencian los dos escenarios? ¿Cómo se traduce eso en tus resultados?

Primero de todos vamos a comparar ambos escenarios, para ver sus respectivas escalas. Primero de todo el ejercicio 1 tiene una cuadrícula con 11 estados posibles, mientras que el ajedrez tiene 63 posibles posiciones para la torre y 58 para la torre, haciendo que hayan $58 \cdot 63 = 3654$ estados posibles. En tema de acciones la torre puede moverse 7 casillas a lo largo y a lo ancho (a veces tiene al rey en medio reduciendo sus posibilidades) y el rey tiene 8 movimientos (con el mismo problema) así que unas 20 posibilidades por turno. El mini agente solamente tiene 3 acciones posibles. En temas de estados terminales tenemos 1 en el ejercicio 1 y 5 en el de ajedrez (todas las columnas finales donde el rey no alcanza la torre). La qtable del escenario 1 es bastante pequeña teniendo únicamente $3 \cdot 11$ valores, la del ajedrez es tan grande que tiene que ser dinámica y su tamaño depende de cada aprendizaje (es muy grande) .

Viendo la dimensión de cada una, podemos deducir que nuestros resultados si se ven afectados. En el primer caso la ejecución es instantánea y siempre obtenemos la política óptima. Encima de eso podemos ver todos sus valores y entender la decisión del agente. Por otro lado, el ajedrez no solo requiere más episodios para poder converger, sino que además la ejecución de cada episodio requiere un tiempo computacional no trivial. Conseguir su política óptima resulta mucho más costoso y lento y encima no podemos entender del todo sus decisiones ya que no tenemos un mapa completo de su qtable (imposible de imprimir completa de forma que se puedan ver y interpretar todos sus valores) .

3.5 Comparación entre el Q-learning y el A estrella en el tablero de ajedrez

3.5.1 Caso determinista

En el caso determinista, el algoritmo A^* funciona correctamente, ya que estamos en un escenario determinista y conocemos el próximo estado que ocurrirá. Con base en eso, podemos encontrar el mejor camino para alcanzar nuestro objetivo. Por otro lado, en este caso de Q-learning, también estamos trabajando en un entorno determinista, donde se pueden definir correctamente las recompensas y los siguientes estados. Aunque hay algunos factores aleatorios, podemos hacer un movimiento determinado descrito en la política con un 100% de certeza.

3.5.2 Caso estocástico

En el caso estocástico, no podemos implementar el algoritmo A^* porque se utiliza para encontrar el mejor camino o forma de obtener la solución, pero no tiene en cuenta los casos aleatorios, como el que se muestra en esta práctica. El caso del marinero borracho, donde tiene un 1% de elegir un movimiento aleatorio, sí puede ser resuelto mediante Q-learning, ya que al explorar, fija resultados en todos los estados por los que pasa. Aunque estos movimientos no sean necesariamente óptimos, permiten al agente acumular conocimiento sobre el entorno, lo que le ayuda a adaptarse a la aleatoriedad. A medida que el marinero sigue explorando y actualizando su política, puede aprender a elegir los movimientos más efectivos a largo plazo, incluso en presencia de incertidumbre. Así, Q-learning se convierte en una herramienta útil para navegar situaciones donde el comportamiento del agente no es completamente predecible.

3.6 Mejores parametros en el Drunken Sailor a la hora de jugar al ajedrez teniendo en cuenta una configuración diferente. ¿Cómo de robusto era tu código anterior?

La forma más simple de comprobar la mejor combinación es simplemente probando varias soluciones. El epsilon dinámico lo dejaremos igual en todos los casos debidos a que explorar al principio y explotar al final es lo más lógico aun teniendo incertidumbre. Lo que cambiaremos serán los valores de alpha y gamma. Préviamente ya habiamos visto como cambiar estos valores nos ayudan a converger (explicado en el punto 3.3.1), pero no habiamos pensado cuales podian ser los mejores. Para eso probaremos 5 sets diferentes.

Recordar que la configuración 2 de la práctica 1 era la siguiente :

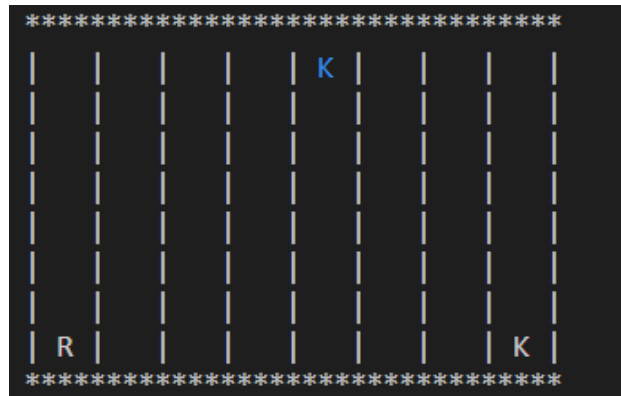


Figura 21: Nuevo escenario

Resultados :

```
Media de episodios en 5 iteraciones Drunked param 1 ( alpha = 0.30 , gamma = 0.90 ): 903.0

Media de episodios en 5 iteraciones Drunked param 2 ( alpha = 0.15 , gamma = 0.90 ): 1076.2

Media de episodios en 5 iteraciones Drunked param 3 ( alpha = 0.30 , gamma = 0.70 ): 809.8

Media de episodios en 5 iteraciones Drunked param 4 ( alpha = 0.15 , gamma = 0.70 ): 892.6

Media de episodios en 5 iteraciones Drunked param 5 ( alpha = 0.10 , gamma = 0.55 ): 857.4
```

Figura 22: Resultado de los nuevos sets

1. Alpha = 0.30 Gamma 0.90 – Media de 903 episodios
2. Alpha = 0.15 Gamma 0.90 – Media de 1076 episodios
3. Alpha = 0.30 Gamma 0.70 – Media de 809 episodios
4. Alpha = 0.15 Gamma 0.70 – Media de 892 episodios
5. Alpha = 0.10 Gamma 0.55 – Media de 857 episodios

En general, observamos que valores más altos de γ (como 0.90) tienden a requerir más episodios para converger, ya que el agente prioriza las recompensas futuras, lo que implica más exploración. Por ejemplo, con $\alpha = 0.15$ y $\gamma = 0.90$, la convergencia ocurre después de 1076 episodios, la mayor cantidad entre las configuraciones probadas. En contraste, valores moderados de γ , como 0.70, permiten un aprendizaje más rápido al balancear recompensas inmediatas y futuras, alcanzando una convergencia más eficiente, como en el caso de $\alpha = 0.30$ y $\gamma = 0.70$ con solo 809 episodios en promedio. Por otro lado, un α bajo (por ejemplo, 0.10) ralentiza el aprendizaje inicial, aunque puede favorecer la estabilidad, como se observa con $\alpha = 0.10$ y $\gamma = 0.55$, donde se necesitan 857 episodios para converger. En conclusión, una combinación equilibrada, como $\alpha = 0.30$ y $\gamma = 0.70$, logra un buen compromiso entre velocidad de aprendizaje y estabilidad a pesar de la incertidumbre. Tener en cuenta también la diferencia de tableros, al requerir un camino más largo, el agente tarda más en aprender al tener más posibilidades de no llegar.

4 Conclusiones y valoraciones personales

Al finalizar la última práctica de IA, hemos visto el concepto de aprendizaje por refuerzo a partir del Q-learning. Hemos trabajado con el algoritmo en dos escenarios diferentes: uno simple, que nos ayudó a entender la idea y cómo hace para encontrar la política óptima, y otro más complejo y realista.

Hemos visto el tiempo que puede tardar en conseguir estos valores para encontrar la política óptima y la importancia de utilizar diferentes recompensas para acelerar el proceso, igual que los otros parámetros. Además, hemos comparado el Q-learning con el algoritmo A* de la P1 y hemos analizado sus diferencias.